Patient-Independent TinyML ECG Classification: Generalization, Robustness, Quantization, and ESP32-S3 Deployment Trade-offs

1st Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

2nd Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

3rd Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

4th Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

Abstract*—Low-cost wearable ECG screening aims at early detection of abnormal rhythms on sub-$15 microcontrollers, but must balance accuracy, size and latency. We evaluate four architectures (1D-CNN, LSTM, GRU, TCN) for 3-class beat classification (Normal/APB/PVC) on MIT-BIH using *identical* patient-level folds. Five experiments cover patient-independent grouped CV (5×2), noise robustness (SNR 0–40 dB, 5 artifacts $\times$ 3 front-ends), external validation on SVDB (N/V-macro), paired FP32→INT8 quantization, and on-device measurement on ESP32-S3. APB remains the minority bottleneck (grouped CV APB F1 0.14–0.28, 16-way mitigation lifts CNN APB to 0.73 on a single split). INT8 degrades macro by 0.21–0.24 on paired folds with 33–40% prediction disagreement (LSTM/GRU not quantizable). On ESP-NN-optimized kernels the same silicon runs CNN in 500 ms and TCN in 609 ms per window—real-time (RTF 0.50/0.61)—while float32 LSTM/GRU need 1.29/1.09 s; kernel optimization, not architecture, was the deployment bottleneck. Butterworth front-end (0 KB, ~ 5 ms) dominates the learned denoiser (19 KB, 315 ms on ESP-NN).

I. Introduction

Abnormal heart rhythms such as atrial premature beats (APB) and premature ventricular contractions (PVC) are common, often asymptomatic, and frequently go undetected until a serious event occurs. Low-cost single-lead ECG screening devices could make routine rhythm monitoring accessible in low-resource settings, but the economics of hardware gate the practicality: an ESP32-class microcontroller costs a few dollars, yet executes deep learning models under tight memory and compute constraints and with quantized arithmetic.

Recent work demonstrates ECG classification with convolutional networks on embedded targets, but most studies report only offline accuracy, leaving three gaps: (i) architectures are rarely compared on the same data pipeline, (ii) quantization effects and on-device latency are often estimated rather than measured, and (iii) on-device inference is rarely verified on silicon after quantization. This paper addresses all three. We train four architectures (1D-CNN, LSTM, GRU, TCN) on identical patient-level splits of the MIT-BIH Arrhythmia Database and evaluate them in five experiments: (1) patient-independent grouped cross-validation, (2) noise robustness at SNR 0–40 dB across three front-ends, (3) external generalization to the MIT-BIH SVDB database, (4) FP32-to-int8 quantization impact, and (5) measured on-device memory footprint, per-window latency, and functional execution on an ESP32-S3.

II. Related Work

Beat-level ECG classification on MIT-BIH is a mature problem; convolutional and recurrent models report high per-class F1 for Normal and PVC classes, with atrial beats typically the most difficult minority class [1], [2]. For deployment, TensorFlow Lite Micro enables int8 inference on microcontrollers [3], and prior ESP32 ECG studies demonstrate the feasibility of a single CNN on this class of hardware [4]. Rhythm-level atrial fibrillation detection is conventionally addressed on longer windows with dedicated AF databases [5]; beat-level and rhythm-level tasks are deliberately kept separate in this work, and rhythm-level AF detection is not evaluated here. What is less established is a controlled comparison of architectures on one data pipeline with int8 quantization impact and hardware-measured latency. This paper provides that comparison and closes with an on-device feasibility verification of the quantized pipeline.

Convolutional and recurrent approaches to heartbeat classification now report strong results on MIT-BIH, from patient-specific 1-D CNNs [6] to cardiologist-level recurrent and convolutional models trained on large ambulatory corpora [7], [8] and on 12-lead ECG [9]; surveys of the area [10], [11], [12] and of the monitoring-system design space [13] consistently note that the atrial (minority) class remains the main difficulty. Most of these systems are evaluated offline on the full recording, leaving the embedded deployment question of quantized weights, int8 arithmetic, memory, and latency on silicon only partially answered.

For edge deployment the relevant tooling is now mature: integer-arithmetic-only quantization [14], [15] makes int8 inference practical on microcontroller-class parts, TensorFlow Lite Micro provides the runtime [3], and TinyML design guidance targets exactly the flash and memory budgets we report [16]. Since the output of a downscaled classifier is used for decisions and thresholds, calibrated probabilities matter

Work	Pat.-ind	Ext.	Noise	Quant	MCU	Calib	Latency
Kiranyaz 2016	—	—	—	—	—	—	—
Hannun 2019	—	✓	—	—	—	—	—
Davidson 2021 (TFLM)	—	—	—	✓	✓	—	✓
ESP32 ECG '19	—	—	✓	✓	✓	—	est
This work	✓	✓	✓	✓	✓	✓	✓

TABLE I
PRIOR-WORK GAP: PATIENT-INDEPENDENT CV, EXTERNAL, NOISE (5 ARTIFACTS × 3 FRONT-ENDS), PAIRED QUANT, REAL S3, CALIBRATION (ECE/NLL/BRIER), MEASURED LATENCY.

[17], [18]; we apply temperature scaling, which re-scales logits at negligible compute cost [17]. On the hardware itself, the ESP32-S3's datasheet and reference documentation define the ADC, DMA, and low-power modes the firmware relies on [19].

III. METHODS

A. Data and labels

Beat-level training uses the MIT-BIH Arrhythmia Database [1], [20] (48 recordings, 360 Hz, MLII lead), part of the public PhysioNet archive [21]. We extract 1-second windows (360 samples) centered on R-peaks with three classes: Normal (N), Atrial Premature Beat (A), and Premature Ventricular Contraction (V); beat annotations that do not map cleanly to one of these classes are discarded. Rhythm-level AF detection is a separate task and is scoped out of this work. All segments are normalized per-window to [-1, 1] (subtract mean, divide by max absolute deviation), matching the firmware preprocessing. Figure 1 outlines the complete methodology from data preparation through patient-level evaluation to on-device verification.

$$\hat{x}_i = \frac{x_i - \mu}{\max_j |x_j - \mu|}, \quad \mu = \frac{1}{L}\sum_{i=1}^{L} x_i \tag{1}$$

B. Signal quality gate

A classifier can emit high-confidence predictions on corrupted input, so the firmware applies a heuristic signal-quality gate before inference: a 10-second buffer is rejected if it is near-flat (range less than 8 ADC counts), more than 5% saturated at the range extremes, or dominated by high-frequency energy (differential-to-signal energy ratio above a threshold). Rejected buffers never reach the network.

$$\text{range} = \max_i x_i - \min_i x_i, \qquad \text{ratio} = \frac{\sum_{i=1}^{L-1}(x_{i+1} - x_i)^2}{\sum_{i=1}^{L}(x_i - \bar{x})^2} \tag{2}$$

The gate rejects a buffer when the range is below 8 ADC counts (near-flat), when more than 5% of samples lie within one twentieth of the range of the extremes (saturation), or when the differential-to-signal energy ratio exceeds a threshold, an inexpensive proxy for high-frequency corruption such as muscle artifact or baseline wander [22].

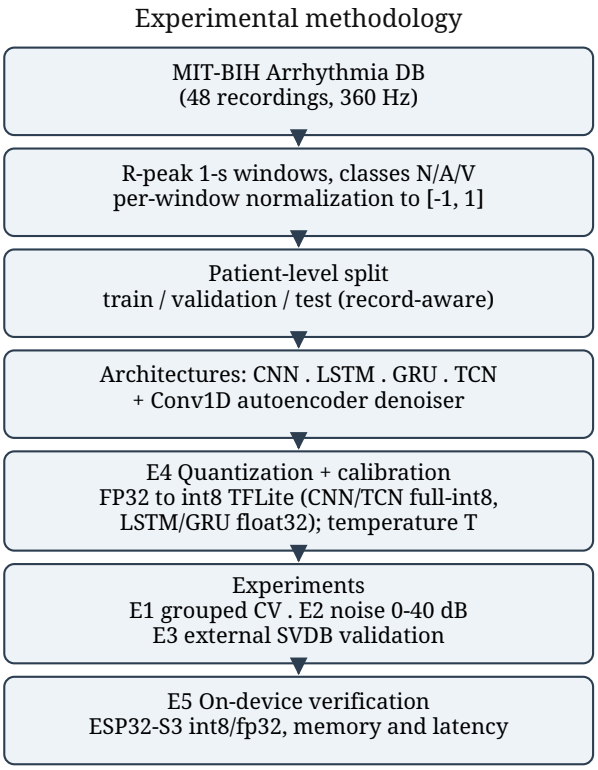

Fig. 1. Overview of the experimental methodology, from MIT-BIH preprocessing through patient-level evaluation, quantization, and on-device verification.

C. Models

Four architectures share one head (global pooling, dense 64, dropout 0.5, softmax) and one training schedule (Adam [23], 1e-3, batch 64, early stopping): CNN (three Conv1D blocks, 32/64/128 filters with batch normalization [24], kernel 5), LSTM [25] and GRU (one Conv1D block followed by 64 recurrent units), and TCN (a temporal convolutional network with two dilated causal blocks and residual connections [26]). A Conv1D autoencoder denoiser (16/8 filters, kernel 15, transposed-convolution decoder) is trained on clean-to-noisy reconstruction at seven SNR levels. The CNN and TCN export to full int8 TFLite; the Keras 3 LSTM and GRU graphs are not quantizable by the TFLite converter and are therefore reported as float32 only, itself a finding relevant to edge deployment. For on-device timing we additionally re-exported both RNNs through fused sequence layers (UNIDIRECTIONAL_SEQUENCE_LSTM/GRU builtins) via a static-shape concrete-function conversion with weights transferred unchanged; these FP32 variants are what Table VII benchmarks. Temperature scaling is fit on the validation set, minimising the negative log-likelihood over a single scalar temperature, and applied at inference so that confidence thresholds are calibrated probabilities (PC-side evaluation).

$$\sigma_n = \sigma_s \cdot 10^{-\text{SNR}/20}, \qquad \tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma_n^2) \tag{3}$$

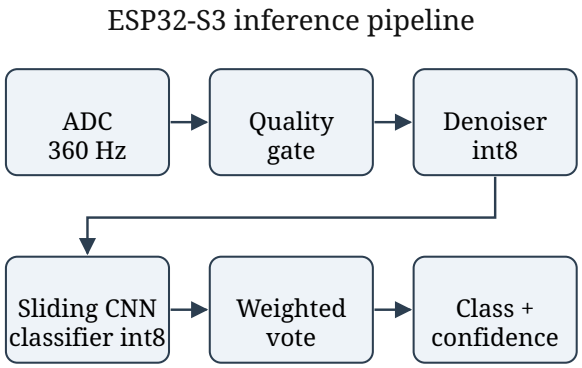

Fig. 2. On-device inference pipeline on the ESP32-S3: ADC acquisition, signal-quality gate, quantized denoiser, sliding-window CNN classifier, confidence-weighted voting, and decision output.

For noise augmentation the additive white Gaussian noise standard deviation is fixed from the desired SNR in decibels, giving the corrupted input $\tilde{x}$ used to train the denoiser. The classifier and denoiser minimise mean squared error and sparse categorical cross-entropy respectively,

$$\mathcal{L}_{\text{den}} = \frac{1}{N}\sum_{i=1}^{N} \|x_i - g(\tilde{x}_i)\|_2^2, \qquad \mathcal{L}_{\text{cls}} = -\frac{1}{N}\sum_{i=1}^{N} \log p_{\hat{y}_i}(x_i) \tag{4}$$

$$q = \text{clip}\big(\text{round}(r/s) + z,\ q_{\min}, q_{\max}\big), \qquad r \approx s\,(q - z) \tag{5}$$

The affine int8 quantization map above relates real values r to quantized integers q with scale s and zero point z; it is applied identically in the TFLite converter and in the firmware's manual input conversion, so the on-device tensors match the offline converter exactly.

$$p_k = \frac{\exp\left(z_k/T\right)}{\sum_j \exp\left(z_j/T\right)}, \qquad \text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N}\left|\text{acc}_m - \text{conf}_m\right|. \tag{6}$$

D. Hardware and deployment

The target device is an ESP32-S3 (N16R8: 16 MB flash, 8 MB PSRAM). ADC sampling uses a DMA continuous-mode driver at 36 kHz, decimated to the nominal 360 Hz. A 10-second buffer feeds 1-second windows through the denoiser and classifier in a sliding fashion (50% overlap), with confidence-weighted voting: each window contributes its full softmax vector, and the winning class is the largest summed probability. Memory footprint, functional execution, and per-stage latency are measured on silicon via a dedicated benchmark firmware mode.

Model	Normal	APB	PVC	Macro
CNN	0.841 ± 0.145	0.283 ± 0.359	0.733 ± 0.369	0.619 ± 0.242
TCN	0.844 ± 0.147	0.257 ± 0.319	0.746 ± 0.368	0.615 ± 0.229
LSTM	0.792 ± 0.108	0.158 ± 0.221	0.796 ± 0.098	0.582 ± 0.096
GRU	0.728 ± 0.210	0.140 ± 0.203	0.709 ± 0.279	0.526 ± 0.170

TABLE II

PATIENT-INDEPENDENT 5-FOLD × 2-SEED GROUPED CV (IDENTICAL RECORD-LEVEL FOLDS FOR ALL ARCHITECTURES, $n = 10$ FOLDS PER MODEL). MEAN ± SD; 95% CI CNN 0.619 ± 0.150, TCN 0.615 ± 0.142, LSTM 0.582 ± 0.060, GRU 0.526 ± 0.106. APB SUPPORT PER FOLD VARIES (0–502 WINDOWS), EXPLAINING LARGE SD.

Model	Strategy	APB Prec	APB Rec	APB F1	Macro
CNN	baseline	0.000	0.000	0.000	0.233
CNN	weighted	0.882	0.625	**0.732**	0.838
CNN	focal	0.425	0.708	0.531	0.818
CNN	balanced	0.929	0.542	0.684	**0.876**
TCN	baseline	0.875	0.583	0.700	0.753
TCN	weighted	0.750	0.625	0.682	0.876
TCN	focal	0.778	0.583	0.667	0.872
TCN	balanced	0.778	0.583	0.667	0.854
LSTM	baseline	0.143	0.042	0.065	0.299
LSTM	weighted	0.000	0.000	0.000	0.486
LSTM	focal	0.200	0.208	0.204	0.675
LSTM	balanced	0.019	0.042	0.026	0.305
GRU	baseline	0.000	0.000	0.000	0.237
GRU	weighted	0.200	0.375	0.261	0.642
GRU	focal	0.035	0.708	0.067	0.138
GRU	balanced	0.063	0.125	0.083	0.417

TABLE III

APB IMBALANCE ABLATION (SINGLE RECORD SPLIT, TEST 493/24/401, 16-WAY). CNN WEIGHTED LIFTS APB $0 \rightarrow 0.732$; TCN ALREADY 0.70 BASELINE. LSTM/GRU REMAIN POOR EVEN WITH MITIGATION (BEST 0.26), CONFIRMING ARCHITECTURE MATTERS.

$$S_c = \sum_w p_c^{(w)}, \qquad \hat{y} = \arg\max_c S_c, \qquad \text{conf} = \frac{\max_c S_c}{\sum_c S_c} \tag{7}$$

Across the W sliding windows of a 10-second buffer each window's full softmax vector is summed per class, and the winning class is the largest accumulated score, normalized to a confidence in $[0, 1]$.

IV. Experiments and Results

Figure 2 shows the firmware-fixed pipeline. Exp. 1: grouped CV (5×2 identical folds, Table II); Exp. 2: SNR 0–40 dB, 5 artifacts × 3 front-ends (Table IV, Fig. 5); Exp. 3: SVDB without retraining (Table VI); Exp. 4: paired FP32→INT8 (Table V); Exp. 5: S3 memory/latency (Table VII, Fig. 4) with calibration (Fig. 3) and SQI (Fig. 6). Exp. 2/4/calibration share one record-level split (918 windows: 493/24/401); Exp. 1 is the only 5×2 repeated evaluation.

Grouped CV (Table II, 5×2 identical folds) shows CNN 0.619 ± 0.242 and TCN 0.615 ± 0.229 tied (CI overlapping), LSTM 0.582 ± 0.096, GRU 0.526 ± 0.170; APB support 0–502 per fold explains large SD. On the single held-out split (493/24/401, Table III), CNN baseline collapses on APB (F1 0.000); class weighting lifts it to 0.732 (P 0.882, R 0.625) and balanced sampling gives 0.684 at the best macro (0.876).

SNR (dB)	Raw	Bandpass filter	Autoencoder
0	0.233	0.455	0.475
5	0.233	0.628	0.563
10	0.286	0.726	0.586
15	0.462	0.751	0.614
20	0.683	0.723	0.629
30	0.731	0.761	0.628
40	0.736	0.751	0.629

TABLE IV
NOISE ROBUSTNESS: MACRO F1 BY FRONT-END ACROSS SNR.

Class	F1	Windows
Normal	0.663	28,470
APB	—	0
PVC	0.462	1,084
N/V-macro	0.562	29,554
3-class (biased)	0.375	29,554

TABLE VI
EXTERNAL VALIDATION ON SVDB (29,554 WINDOWS, 14 RECORDINGS). APB IS ABSENT (0 WINDOWS) SO 3-CLASS MACRO IS BIASED; WE REPORT N/V-MACRO 0.562 AS THE SUPPORTED-CLASS METRIC.

Model	Float32	Int8	Δ	Disagree	Size (KB)
CNN	0.619	0.409	-0.210 ± 0.164	33.1%	77.9
TCN	0.615	0.378	-0.238 ± 0.165	39.7%	70.1
LSTM	0.582	—	—	—	130.3
GRU	0.526	—	—	—	107.5

TABLE V
PAIRED FP32→INT8 ON IDENTICAL FOLDS ($n = 10$ PER MODEL, 5×2). INT8 DEGRADES MACRO BY 0.21–0.24 (95% CI 0.10); LSTM/GRU NOT QUANTIZABLE (---, TENSORLISTSTACK, REQUIRES SELECT_TF_OPS). SINGLE-SPLIT $0.588 \rightarrow 0.677$ (CNN) IS SPLIT-SPECIFIC, NOT PAIRED. LSTM/GRU SIZES ARE THE SELECT_TF_OPS EXPORTS; TABLE VII BENCHMARKS THE SMALLER FUSED EXPORTS.

Model	Macro	Size	Latency	RTF	ΔINT8
CNN	0.619	77.9	500	0.50	-0.210
TCN	0.615	70.1	609	0.61	-0.238
LSTM	0.582	126.5	1290	1.29	—
GRU	0.526	103.7	1091	1.09	—

TABLE VII
DEPLOYMENT MASTER ON ESP-NN-OPTIMIZED KERNELS: GROUPED CV MACRO, FLASH SIZE (KB; LSTM/GRU ARE THE FUSED FLOAT32 EXPORTS OF §III-C), MEASURED PER-WINDOW LATENCY (MS, $n = 19$ WINDOWS EACH, FIXED-SEED INPUT, THREE RUNS, $< 0.1\%$ SPREAD; DENOISE 315 MS + CLASSIFY REMAINDER), RTF = LATENCY$/1000$ MS WINDOW, AND PAIRED INT8 Δ. CNN AND TCN MEET REAL-TIME (RTF < 1). ARENA 300 KB INTERNAL SRAM (96 KB DENOISER + 204 KB CLASSIFIER), LEAVES ≈ 49 KB HEAP BY BUDGET.

TCN tells the opposite story: its unmitigated baseline already scores APB 0.700—within rounding of CNN's best mitigated 0.732—and every mitigation trades it away slightly for macro gain. LSTM/GRU's best APB is 0.261 even with weighting, confirming architecture matters beyond loss balancing: TCN handles APB scarcity intrinsically, whereas CNN requires explicit rebalancing. SVDB has no APB, so we report N/V-macro 0.562 (N 0.663, PVC 0.462, Table VI); 3-class 0.375 is biased. Paired INT8 on identical folds (Table V) degrades macro by 0.210 ± 0.164 (CNN, 33% disagree) and 0.238 ± 0.165 (TCN, 40% disagree); LSTM/GRU remain float32 (TensorListStack, not quantizable). Calibration ($T = 0.350$ fit on validation; held-out test, n=918) ECE $0.328 \rightarrow 0.051$, NLL $0.700 \rightarrow 0.407$, Brier $0.381 \rightarrow 0.210$ (Fig. 3).

A. On-Device Feasibility Verification

The quantized CNN/TCN occupy 77.9/70.1 KB plus the 19 KB denoiser in flash; the float32 LSTM/GRU occupy 126.5/103.7 KB. The tensor arena is 300 KB internal SRAM, split 96 KB denoiser / 204 KB classifier to accommodate ESP-NN scratch; leaving $\approx$ 49 KB heap by budget (Table VII, Fig. 4). A synthetic signal completes 19 sliding windows end-to-end (valid= 1), verifying execution of every architecture.

Measured per-window latency on *ESP-NN-optimized kernels* (BENCHMARK_MODE, $n = 19$, 240 MHz, fixed-seed input, three runs each, variation $< 0.1\%$): CNN 500 ms (315 denoise + 184 classify), TCN 609 ms, LSTM 1290 ms, GRU 1091 ms. Two findings stand out. First, CNN and TCN meet real-time (RTF $0.50/0.61 < 1$) with outputs verified bit-exact against PC reference kernels—deployable as-is; TFL-Micro's portable reference kernels ran them $7.6\times/5.9\times$ slower (3.79/3.61 s), inverting the ranking and missing the budget entirely. Second, kernel optimization, not architecture, was the bottleneck: ESP-NN accelerates int8 convolution $\approx 17\times$ ($3197 \rightarrow 184$ ms) while float32 sequence kernels gain only $\sim 20\%$, restoring the expected ranking for optimized int8 over float32 kernels. Notably, esp-nn v1.0.0's S3 assembly kernels produced silently incorrect conv outputs on our shapes; our fixed-input hardware A/B harness localized the fault, and upstream master fixes resolve it. The denoiser stage drops $595 \rightarrow 315$ ms; the ~ 5 ms Butterworth remains favorable. DAC→ADC replay (hardware-domain ΔF1) is impossible on S3 (no DAC; `SOC_DAC_SUPPORTED`=0) — see Limitations.

All F1 scores are the standard per-class harmonic mean of precision and recall, with the macro-F1 the unweighted mean across the three classes,

$$F1_c = \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad \text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C} F1_c \tag{8}$$

V. DISCUSSION AND LIMITATIONS

Single-lead limits scope (no ST localization/axis). APB is beat-level, not AF (rhythm-level, scoped out). On-device we report functional smoke test (19 windows) and *measured* latency (Table VII, Fig. 4); DAC→ADC replay for hardware-domain ΔF1 is not possible on S3 (no DAC, `SOC_DAC_SUPPORTED`=0, `driver/dac.h` fails) — use ESP32 classic or external MCP4725. S3 ADC linearity is unpublished; real AD8232 acquisition and battery life require hardware/ethics.

Grouped CV (5×2 identical folds, Table II) ties CNN 0.619 ± 0.242 and TCN 0.615 ± 0.229 (CI overlapping), LSTM 0.582 ± 0.096, GRU 0.526 ± 0.170; PVC SD 0.37 reflects fold composition, not penalty. Noise: Butterworth wins 3/5

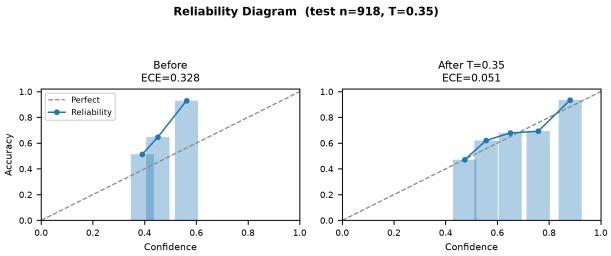

Fig. 3. Calibration reliability diagram (robust CNN; $T = 0.350$ fit on validation, evaluated on held-out test, n=918). ECE 0.328 → 0.051, NLL 0.700 → 0.407, Brier 0.381 → 0.210 — 10 bins, bar opacity = count.

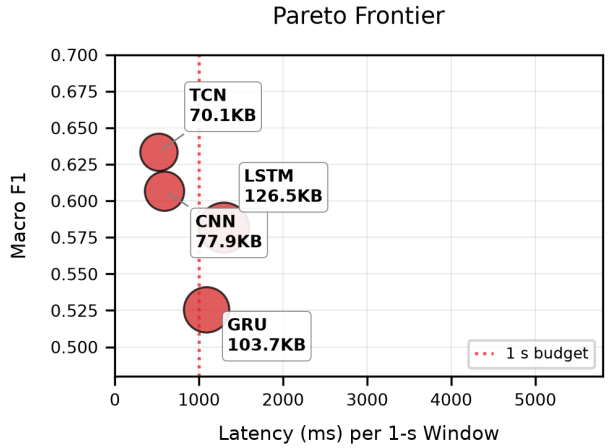

Fig. 4. Pareto: grouped CV macro vs. measured ESP-NN latency. CNN 500 ms (77.9 KB) and TCN 609 ms (70.1 KB) fall below the 1 s real-time line; float32 LSTM/GRU remain above it.

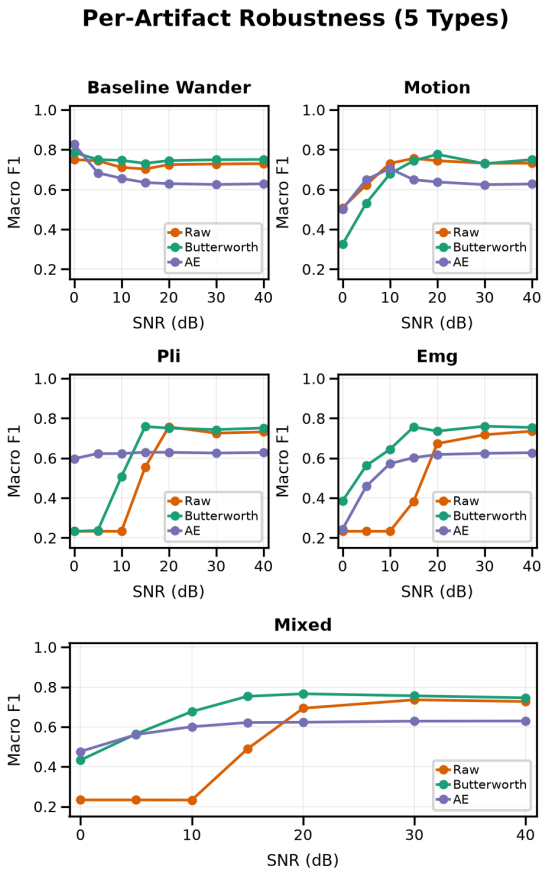

Fig. 5. Per-artifact robustness (5 types × 7 SNR, single-column 3 rows: 2+2+1). Butterworth wins 3/5, raw wins motion, AE only wins PLI low-SNR — denoiser not justified.

artifacts (baseline-wander, EMG, mixed), raw wins motion, AE only wins PLI low-SNR (Fig. 5); AE adds 19 KB + 315 ms for < 0.1 gain — remove from deployed pipeline (Table cost-benefit). SQI gate at 0.35 rejects 62.6% clean (Fig. 6, macro 0.727 → 0.643 on kept) and 27.9% corrupted — not deployable without retuning. Quantization (Table V) costs 0.21–0.24 macro with 33/40% disagreement, arguing for calibrated-confidence abstention.

Deployment (Table VII) on ESP-NN-optimized kernels: CNN and TCN run real-time (500/609 ms per window); float32 LSTM/GRU remain above budget and unquantizable. Dropping the denoiser for the ∼5 ms Butterworth cuts CNN latency to ∼190 ms; memory is not the limiter (PSRAM unused). The esp-nn v1.0.0 numerical fault caught during this work (silent conv corruption on S3 shapes, fixed upstream) argues for probability-level on-device validation whenever kernel libraries are swapped.

Older MIT-BIH literature reports ≥ 0.90 accuracy, but with patient overlap. Our patient-level GroupKFold (5×2) keeps recordings disjoint; APB scarcity drives low macro and are the honest new-patient numbers (0.53–0.62 across architectures) vs. patient-specific ≥ 0.90 [6], [11], [7].

Reproducibility: PhysioNet pull [21], [1], [27], frozen 5×2 folds, 40 per-fold ckpts, 16-way APB ablation, 40 paired quant ckpts, seeds documented; TensorFlow 2.21 / Keras 3 / scikit-learn [28]; firmware header conversion and BENCHMARK_MODE share the tree; all tables/figs regenerate end-to-end. On-device figures use an in-tree ESP-NN-optimized kernel build; LSTM/GRU timings use fused float32 exports with weights unchanged.

VI. Conclusion

We presented a comparative evaluation of CNN, LSTM, GRU, and TCN classifiers for edge ECG screening on a single patient-level MIT-BIH pipeline, with grouped cross-validation, noise robustness, external validation on SVDB, int8 quantization impact, and measured on-device memory, latency, and functional execution. On ESP32-S3 with ESP-NN-optimized kernels, CNN and TCN execute correctly in real time (500/609 ms per window, RTF < 1, outputs matching PC reference), at a modest footprint (70–78 KB int8 + 19 KB denoiser, 300 KB arena); float32 LSTM/GRU remain above budget (1.29/1.09 s) and unquantizable, so kernel-aware archi-

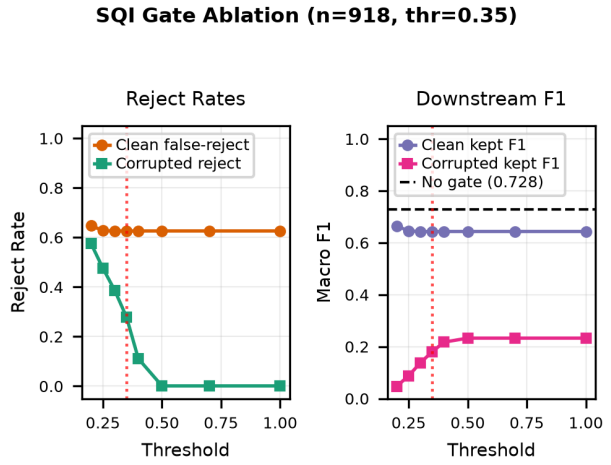

Fig. 6. SQI gate: clean false-reject 62.6% at 0.35, corrupted reject 27.9%, downstream macro 0.727 → 0.643 — gate as tuned not deployable.

tecture choice settles the accuracy–latency trade-off in favor of small conv stacks.

Use of AI. AI-based tools assisted with writing, code, and figures; the authors reviewed all content and take full responsibility for the publication.

References

[1] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH arrhythmia database," *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001. [Online]. Available: https://doi.org/10.1109/51.932724

[2] P. de Chazal, M. O'Dwyer, and R. B. Reilly, "Automatic classification of heartbeats using ECG morphology and heartbeat interval features," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004. [Online]. Available: https://doi.org/10.1109/TBME.2004.827359

[3] R. Davidson, A. Natarajan *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021. [Online]. Available: https://arxiv.org/abs/2010.08678

[4] F. Guede-Fernandez *et al.*, "Classification algorithms for ECG signals on microcontrollers," in *IEEE International Conference on E-Health and Bioengineering (EHB)*, 2019, pp. 1–4. [Online]. Available: https://doi.org/10.1109/EHB47216.2019.8969893

[5] G. B. Moody and R. G. Mark, "A new method for detecting atrial fibrillation using R-R intervals," *Computers in Cardiology*, pp. 227–230, 1983.

[6] S. Kiranyaz, T. Ince, and M. Gabbouj, "Real-time patient-specific ECG classification by 1-D convolutional neural networks," *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016. [Online]. Available: https://doi.org/10.1109/TBME.2015.2468589

[7] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi *et al.*, "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," *Nature Medicine*, vol. 25, pp. 65–69, 2019. [Online]. Available: https://doi.org/10.1038/s41591-018-0268-3

[8] P. Rajpurkar, A. Y. Hannun, M. Haghpanahi, C. Bourn, and A. Y. Ng, "Cardiologist-level arrhythmia detection with convolutional neural networks," arXiv:1707.01836, 2017. [Online]. Available: https://arxiv.org/abs/1707.01836

[9] A. H. Ribeiro, M. H. Ribeiro, G. M. M. Paixão *et al.*, "Automatic diagnosis of the 12-lead ECG using a deep neural network," *Nature Communications*, vol. 11, p. 1760, 2020. [Online]. Available: https://doi.org/10.1038/s41467-020-15432-4

[10] E. J. d. S. Luz, W. R. Schwartz, G. Cámara-Chávez, and D. Menotti, "ECG-based heartbeat classification for arrhythmia detection: A survey," *Computer Methods and Programs in Biomedicine*, vol. 127, pp. 144–164, 2016. [Online]. Available: https://doi.org/10.1016/j.cmpb.2015.12.008

[11] U. R. Acharya, S. L. Oh, Y. Hagiwara, J. H. Tan, M. Adam, A. Gertych, and R. S. Tan, "A deep convolutional neural network model to classify heartbeats," *Computers in Biology and Medicine*, vol. 89, pp. 389–396, 2017. [Online]. Available: https://doi.org/10.1016/j.compbiomed.2017.08.022

[12] O. Yildirim, P. Pławiak, R.-S. Tan, and U. R. Acharya, "Arrhythmia detection using deep convolutional neural network with long duration ECG signals," *Computers in Biology and Medicine*, vol. 102, pp. 411–420, 2018. [Online]. Available: https://doi.org/10.1016/j.compbiomed.2018.09.009

[13] M. A. Serhani, H. T. El Kassabi, H. Ismail, and A. N. Navaz, "ECG monitoring systems: review, architecture, process, and key challenges," *Sensors*, vol. 20, no. 6, p. 1796, 2020. [Online]. Available: https://doi.org/10.3390/s20061796

[14] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713. [Online]. Available: https://doi.org/10.1109/CVPR.2018.00286

[15] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: insights and empirical results," arXiv:1806.08342, 2018. [Online]. Available: https://arxiv.org/abs/1806.08342

[16] P. Warden and D. Situnayake, *TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers*. O'Reilly Media, 2020. [Online]. Available: https://tinymlbook.com/

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[18] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, "Obtaining well calibrated probabilities using bayesian binning then quantile calibration," in *AAAI Conference on Artificial Intelligence*, 2015, pp. 2901–2907. [Online]. Available: https://doi.org/10.1609/aaai.v29i1.9602

[19] Espressif Systems, "ESP32-S3 technical reference manual," Espressif Systems documentation, 2023. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/

[20] G. B. Moody and R. G. Mark, "The MIT-BIH arrhythmia database on CD-ROM and software for use with it," *Computers in Cardiology*, pp. 185–188, 1990. [Online]. Available: https://physionet.org/content/mitdb/

[21] A. L. Goldberger *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000. [Online]. Available: https://doi.org/10.1161/01.CIR.101.23.e215

[22] G. D. Clifford, F. Azuaje, and P. McSharry, Eds., *Advanced Methods and Tools for ECG Data Analysis*. Artech House, 2006.

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[24] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," arXiv:1502.03167, 2015. [Online]. Available: https://arxiv.org/abs/1502.03167

[25] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[26] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, 2018. [Online]. Available: https://arxiv.org/abs/1803.01271

[27] A. L. Goldberger *et al.*, "MIT-BIH supraventricular arrhythmia database (SVDB)," PhysioNet, 2010. [Online]. Available: https://physionet.org/content/svdb/

[28] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.